CLOUDPULSE: A CLOUD COST MONITORING AND OPTIMISATION PLATFORM WITH OWNERSHIP ATTRIBUTION AND VERIFIED SAVINGS

A PROJECT REPORT

Submitted by

[REGISTER NUMBER] **[STUDENT NAME]**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

R. M. K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

PUDUVOYAL



OCTOBER 2026

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution)
R.S.M. Nagar, PUDUVOYAL-601 206
Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

BONAFIDE CERTIFICATE

This is to certify that **Mr./Ms. _____**, bearing **Register Number _____**, of __ **Year**, __ **Semester**, **Department of Computer Science and Engineering**, has successfully completed the mini project entitled **"CloudPulse: A Cloud Cost Monitoring and Optimisation Platform with Ownership Attribution and Verified Savings"** during the academic year **2026–2027** under my guidance and supervision.

The work submitted in this report is a bonafide record of the work carried out by the student/team as part of the academic requirements of the course.

Name of the Student **Project Guide**

ABSTRACT

Cloud spending is easy to start and hard to control. Organisations running workloads on Microsoft Azure and Amazon Web Services routinely pay for resources that nobody uses: virtual machines shut down the wrong way that still bill for compute, disks and snapshots left behind when a project ends, and public IP addresses attached to nothing. Most cost tools stop at a dashboard that reports how much was spent, leaving the decisions about who should act, and whether the action helped, to people. This project presents CloudPulse, a cloud cost monitoring and optimisation platform that carries each cost problem through a complete loop: signal, context, finding, ownership, action and verification. CloudPulse signs in to an Azure subscription with a read-only service principal holding only the Reader and Cost Management Reader roles, and reads the bill, resource inventory, virtual machine metrics, Azure Advisor recommendations and the Activity Log through a small custom REST client. Rule-based detectors and transparent statistics, namely a weekday-aware robust z-score for spend anomalies, Holt's linear smoothing for month-end and budget forecasts, and a percentile verdict for idle and oversized machines, turn this data into de-duplicated findings. Each finding is priced per month, scored from P1 to P4, assigned to the person who created the resource, and tracked through a defined lifecycle. A saving is counted only after seven days of billed cost confirm the drop, and a fixed resource that returns reopens its finding. The system uses Python, FastAPI, SQLAlchemy and a framework-free JavaScript interface, encrypts stored cloud secrets with Fernet, and is validated by 27 automated tests and a scripted Azure environment that plants real waste. On the bundled FOCUS sample bill CloudPulse reported $262.94 a month of open waste and $33.44 a month of verified savings at 94% estimate accuracy.

TABLE OF CONTENTS

S.No	Section	Page No.
1	Introduction	1
2	Objectives	5
3	Tools and Technologies	8
4	Architecture Diagram	15
5	Methodology	22
6	Results	30
7	Conclusion	39
8	Reference	41

LIST OF FIGURES AND TABLES

Figures

Figure 4.1 CloudPulse system architecture with the Azure credential and data flows 17
Figure 4.2 Azure credential creation, token acquisition and read-only request sequence 19
Figure 5.1 The synchronisation pipeline executed on every sync 23
Figure 5.3 Priority score against monthly impact (confidence 0.9, new finding), produced by the implemented function 26
Figure 5.2 Finding lifecycle, with automatic and manual transitions 27
Figure 6.1 CloudPulse sign-in page (127.0.0.1:8000/login) 30
Figure 6.2 Overview page (administrator view of the demo workspace) 31
Figure 6.3 Cost explorer with the daily tooltip for 18 September 2024 33
Figure 6.4 Anomalies page showing the flagged day 24 September 2024 34
Figure 6.5 Budgets page with the cumulative spend line and the budget breach marker 35

Tables

Table 1.1 Capability comparison of existing approaches and CloudPulse 3
Table 2.1 Functional requirements 6
Table 2.2 Non-functional requirements 6
Table 3.1 Development and run-time environment 8
Table 3.2 Azure services, interfaces and the permission they need 9
Table 3.3 Main Python dependencies (requirements.txt) 10
Table 3.4 Cloud computing modules and where CloudPulse applies them 14
Table 4.1 Numbered flows in Figure 4.1 18
Table 4.2 Three ways CloudPulse can sign in to Azure 18
Table 4.3 Database tables 20
Table 4.4 Threats and the controls that address them 21
Table 5.1 Waste detection rules applied to the Azure inventory 24
Table 5.2 Azure scripts 28
Table 5.3 Waste planted by 02-leaks.sh and the finding CloudPulse raises 28
Table 5.4 Automated tests (27) 29
Table 6.1 Set-up used for the results in this chapter 30
Table 6.2 Findings in the demo workspace (impact in USD per month) 36
Table 6.3 Verified savings in the demo workspace 36
Table 6.4 Automated test run (pytest) 37
Table 6.5 Validation checklist for a live subscription 37

1. INTRODUCTION

1.1 Background

Cloud computing has changed how organisations buy technology. Instead of purchasing servers in advance, a team can create a virtual machine, a database or a storage account in minutes and pay only for the time and capacity it uses. Platforms such as Microsoft Azure and Amazon Web Services (AWS) bill by the second or by the hour, and every resource created from a command line, a portal or an automation script starts to accumulate cost from the moment it exists [1]. This elasticity is the main strength of the cloud, but it is also its main financial risk: resources are easy to create and easy to forget [20].

Forgotten resources are the most common source of avoidable cloud spend. A virtual machine that was shut down from inside the operating system is reported as stopped, yet the platform still reserves the hardware for it and keeps billing for compute. A data disk that was detached when a project ended continues to be charged for every gigabyte it holds. A static public IP address that no network interface uses, and a snapshot whose source disk was deleted months ago, each add a small amount every day. None of these items is large on its own, but across hundreds of resources and many teams they add up to a significant monthly figure that delivers no value.

The discipline that deals with this problem is called **FinOps** (cloud financial operations). The FinOps Foundation describes it as a cultural practice in which engineering, finance and business teams share responsibility for cloud spend and make timely, data-driven decisions [2]. A central idea of FinOps is that visibility alone is not enough: a cost problem is only solved when somebody who owns it acts and the saving can be shown on the next bill.

1.2 Problem Statement

Native cost consoles, such as Azure Cost Management and AWS Cost Explorer, are very good at answering "how much did we spend and on what?". They are much weaker at answering the questions that decide whether money is actually saved:

- **Which resources are wasted?** A list of the most expensive services does not say which individual resources are idle.
- **Who created each wasted resource?** Resource tags are often missing, so the engineer who can switch the resource off is unknown.

- **What should be fixed first?** Without a single score that combines money, severity and confidence, every item looks equally urgent.
- **Was it really fixed?** A closed ticket is not evidence. Only a lower bill proves a saving.
- **Did the waste come back?** Resources that are recreated after a clean-up silently erase the benefit.

The problem addressed by this project is therefore to build a system that does not stop at reporting cost, but carries each cost problem through detection, prioritisation, ownership, action and independent verification, while remaining safe to connect to a real cloud account.

1.3 Motivation

The motivation for the project came from three observations. First, a student or small team that experiments on Azure, for example with the free credit of Azure for Students, can exhaust the credit with a handful of forgotten resources, and has no tool that explains why. Second, the algorithms that good FinOps tools use, such as robust anomaly detection and forecasting, are rarely explained: the user sees a number but cannot trace how it was produced. Third, security is often an afterthought; many tools ask for broad write permissions that an organisation should never hand to a third party. The project aims to show that a transparent, read-only and verifiable cost platform can be built with a small, understandable code base.

1.4 Existing Systems and Their Limitations

Three families of tools are commonly used. **Native consoles** (Azure Cost Management, Azure Advisor, AWS Cost Explorer) are accurate and free, but each covers one provider and stops at reporting or a generic recommendation. **Commercial FinOps platforms** add multi-cloud views, allocation and dashboards, but are closed products whose scoring methods are not visible, and which generally require a paid subscription and broad access. **Home-made scripts** can list idle resources, but they produce one-off reports with no workflow, no history and no verification. Table 1.1 compares the capabilities that matter for the loop described above.

Table 1.1: Capability comparison of existing approaches and CloudPulse

Capability	Native consoles	Commercial FinOps tools	Ad-hoc scripts	CloudPulse
Spend dashboards and breakdown	Yes	Yes	Rarely	Yes
Idle and oversized resource detection	Partial (Advisor)	Yes	Partial	Yes
Creator of each resource from audit log	Manual lookup	Varies	No	Yes, automatic
Task workflow (assign, start, fix, dismiss)	No	Varies	No	Yes
Single priority score (P1 to P4)	No	Varies	No	Yes, explainable
Saving checked against the next bill	No	Rarely	No	Yes, 7-day before/after
Saving withdrawn if waste returns	No	Rarely	No	Yes
Read-only access to the account	n/a	Varies	Depends	Yes, by design
Transparent, documented algorithms	Partial	Usually no	Yes	Yes, open code

1.5 Proposed System

CloudPulse is a cloud cost monitoring and optimisation platform built around one idea: every cost problem should travel through the same loop, from the first signal to a verified result. The loop has six stages:

1. **Signal.** On every sync CloudPulse reads the bill, the resource inventory, virtual machine metrics, Azure Advisor recommendations and the Activity Log.
2. **Context.** Each signal is priced per month, using the actual billed cost of the resource where it exists and published list prices otherwise.
3. **Finding.** The signal becomes one de-duplicated, scored task with its evidence attached.
4. **Ownership.** The creator found in the Activity Log (tags second) is matched to a CloudPulse user and the finding is assigned.
5. **Action.** The owner starts work, fixes, dismisses with a reason or snoozes, with notifications and a full history.
6. **Verification.** CloudPulse confirms the problem has disappeared and compares seven days of billed cost before and after. Only then is the saving recorded.

CloudPulse connects to an Azure subscription through a read-only service principal, an Azure CLI login or an App Service managed identity. It never creates, changes or deletes anything in the cloud account.

1.6 Scope of the Project

The project delivers a working web application with a Python back end and a browser front end, an Azure test environment that can be created with scripts, and an automated test suite. The following boundaries apply:

- Azure is the primary supported cloud. AWS (through boto3) and bills in the FinOps **FOCUS** open format are supported as optional inputs.
- CloudPulse is **read-only**. It recommends fixes (for example the exact `az vm deallocate` command) but never executes them.
- Notifications are in-application only. E-mail and chat integrations are listed as future work.
- Savings figures are limited to resource findings; anomalies, budget risks and tagging gaps are tracked and verified but are not counted as savings.

1.7 Organisation of the Report

Chapter 2 states the objectives and requirements. Chapter 3 lists the tools and technologies and the cloud computing concepts applied. Chapter 4 presents the system architecture, including how Azure credentials are created, stored and used. Chapter 5 explains the methodology: data collection, detection rules, the anomaly, forecast, priority and verification algorithms, the finding lifecycle and the test environment. Chapter 6 shows and explains the results with screenshots of the running application. Chapter 7 concludes the report and Chapter 8 lists the references.

2. OBJECTIVES

2.1 Aim

To design and implement a transparent, read-only cloud cost monitoring and optimisation platform that detects wasted spend in a real Azure subscription, assigns each problem to the person who created the resource, tracks the fix, and counts a saving only after the bill proves it.

2.2 Specific Objectives

1. To collect cost, inventory, usage, recommendation and audit data from an Azure subscription using the Azure Resource Manager (ARM) REST interface and a least-privilege identity.
2. To detect idle and wasted resources: unattached disks, unused public IP addresses, orphaned snapshots, stopped-but-billed virtual machines, running-but-idle machines and oversized machines.
3. To detect abnormal daily spend with a robust, weekday-aware statistical method and to explain the services that caused it.
4. To forecast month-end spend with an uncertainty range and to predict the day a budget will be exceeded.
5. To attribute every resource to its creator using the Azure Activity Log, falling back to owner tags.
6. To turn all signals into a single ranked list of findings, with a score from 0 to 100 and a band from P1 to P4 that can be explained term by term.
7. To manage each finding through a defined lifecycle with role-based permissions, comments, notifications and an audit trail.
8. To verify every claimed saving against billed cost and to withdraw the saving if the problem returns.
9. To protect stored cloud credentials and user passwords with modern cryptography and to keep the application safe to expose on the internet.
10. To provide a reproducible Azure test environment, with intentionally planted waste, and an automated test suite.

2.3 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement	Where implemented
FR1	Sign in with e-mail and password; forced password change for temporary passwords; one-click read-only demo	routers/auth.py, security.py
FR2	Create customer workspaces; connect Azure (service principal, CLI or managed identity) and optionally AWS	routers/admin.py, services/connections.py
FR3	Read daily cost by service and by resource	azure_collector.py (Cost Management)
FR4	Read inventory and VM power state; read VM CPU/network metrics	Resource Graph, Azure Monitor
FR5	Detect waste and rank it by priority	azure_collector.py, analytics.py
FR6	Find the creator of each resource	owners.py (Activity Log, tags)
FR7	Spend anomalies with drivers; month forecast with range; budget breach date	analytics.py, workspace_detectors.py
FR8	Finding workflow: assign, start, resolve, dismiss, snooze, reopen, comment	findings.py
FR9	7-day before/after saving verification and a savings ledger	findings.verify, analytics.before_after
FR10	Import bills in the FOCUS format; set a monthly budget	focus_importer.py, api.py
FR11	Notifications, audit log and CSV export of findings	api.py, audit.py

2.4 Non-Functional Requirements

Table 2.2: Non-functional requirements

Quality	Requirement
Security	Least-privilege read-only access; secrets encrypted at rest; scrypt password hashes; HttpOnly session cookies; CSRF header; login rate limit; security headers
Reliability	Retry on HTTP 429 and 5xx with Retry-After; resumable paging; one failed data source must not stop a sync
Explainability	Every score and forecast can be recomputed by hand from the evidence stored with the finding
Portability	Runs on a laptop, in Docker, on Render or on Azure App Service; SQLite by default, PostgreSQL ready
Usability	No front-end framework to install; responsive from 400 px phones to wide desktops; light and dark themes
Maintainability	Algorithms are pure functions with unit tests; services are separated from routers
Cost	Runs on the free tier of a hosting service; the Azure test workload costs about $2.3 per day

2.5 Success Criteria

- All automated tests pass (27 tests in the final version).
- In demo mode the application shows consistent figures across every page: spend, open waste, verified savings and budget status.
- Against a real Azure subscription, planted waste (a disk, an IP, a snapshot and a stopped VM) appears as findings with the creator identified, without any tags.

- A fixed finding becomes verified only after billed data confirms a drop.

2.6 Constraints and Assumptions

Azure publishes cost data 8 to 24 hours after usage, so cost charts and verification always lag the actual change. Azure for Students limits the regions that can be used, which the scripts handle by reading the subscription's allowed-location policy. Memory utilisation of virtual machines is not available without an in-guest agent, so oversized-machine findings are given lower confidence. The Activity Log keeps 90 days of events, so the creator of an older resource may be unknown unless a tag exists.

3. TOOLS AND TECHNOLOGIES

3.1 Hardware and Software Requirements

Table 3.1: Development and run-time environment

Item	Specification
Processor / memory	Any 64-bit CPU, 2 GB RAM or more (the application itself uses well under 500 MB)
Operating system	Windows 10/11, Linux or macOS for development; Linux container for deployment
Language run-time	Python 3.10 or newer (Python 3.12 in the Docker image)
Browser	Any current Chromium, Firefox or Safari browser
Cloud account	A Microsoft Azure subscription (Azure for Students is sufficient)
Command-line tools	Azure CLI (or Azure Cloud Shell), Git, Docker (optional)
Network	Outbound HTTPS to management.azure.com, login.microsoftonline.com and prices.azure.com

3.2 Back-End Technologies

Python was chosen because it has mature libraries for statistics, cryptography and cloud access and because its readable syntax suits an explainable algorithms layer. The web service is built with **FastAPI** [3], an asynchronous framework that validates request bodies with **Pydantic** and serves the JSON API and the static front end from one process, run by the **Uvicorn** server. Persistence uses **SQLAlchemy 2.0** [4], an object-relational mapper that lets the same models run on **SQLite** during development and on **PostgreSQL** in production.

Background work is handled by **APScheduler**, which starts a synchronisation of every workspace a few seconds after start-up and then at a configurable interval (360 minutes by default). Long operations run in a worker thread so the server keeps answering requests. Secrets are protected with the **cryptography** package: the Fernet recipe for stored cloud credentials and the standard library's **scrypt** for passwords [5], [6].

3.3 Cloud Access Libraries

Authentication to Azure uses the **azure-identity** library, but only for the credential classes (`ClientSecretCredential`, `ManagedIdentityCredential` and `AzureCliCredential`) that obtain OAuth 2.0 access tokens. All data calls are made by a custom ARM REST client of about 135 lines that uses only the standard library. The reason is practical: the Azure management SDKs pull in dozens of dependencies, while the five APIs CloudPulse needs (Cost Management, Resource Graph, Monitor metrics, Activity Log and Advisor [12]-[16]) share the same simple shape,

`https://management.azure.com` with a Bearer token and an `api-version` parameter. The custom client adds the behaviour a monitoring tool needs: waiting for `Retry-After` on HTTP 429 and 5xx, following `nextLink` and `$skipToken` paging, and returning exact error messages. **boto3** is used for the optional AWS collector.

3.4 Front-End Technologies

The user interface is a single-page application written in plain **HTML, CSS and JavaScript** with no framework or build step. This keeps the deployment to a folder of static files, loads instantly and is easy to read. All charts (stacked daily bars, the cumulative budget line with a forecast band, before/after bars and sparklines) are drawn as **SVG** by a 290-line module, so no charting library is needed. Typography uses the self-hosted IBM Plex and Source Serif fonts under the SIL Open Font License. The design system uses CSS variables for light and dark themes and breakpoints at 560, 900 and 1180 pixels for responsive layout.

3.5 Azure Services and Interfaces Used

Table 3.2: Azure services, interfaces and the permission they need

Service	Interface used	Purpose in CloudPulse	Permission
Microsoft Entra ID	login.microsoftonline.com/ {tenant}/oauth2/v2.0/token	Issue a Bearer token for the service principal	App registration + secret
Cost Management	Microsoft.CostManagement/query (ActualCost, daily)	Daily cost by service and by resource	Cost Management Reader
Resource Graph	Microsoft.ResourceGraph/ resources (KQL)	Disks, public IPs, snapshots, VMs, tags	Reader
Azure Monitor	Microsoft.Insights/metrics	Hourly CPU and network of each VM for 14 days	Reader
Activity Log	Microsoft.Insights/eventtypes/ management	Who created each resource (90 days)	Reader
Azure Advisor	Microsoft.Advisor/ recommendations (Cost)	Microsoft's own cost recommendations	Reader
Retail Prices API	prices.azure.com/api/retail/prices	List-price fallback; no sign-in needed	None (public)

3.6 Data Set

For the demonstration mode the project bundles a 100,000-record sample of the FinOps **FOCUS** (FinOps Open Cost and Usage Specification) format [7]. It is real, anonymised billing data of an organisation using several cloud providers. CloudPulse aggregates it into 1,166 daily cost rows from 21 March to 30 September 2024. FOCUS is also the import format for customers who prefer to upload a bill instead of connecting an account.

3.7 DevOps and Testing Tools

- **Docker and Docker Compose** package the application with a persistent volume for the database.
- **Render** (`render.yaml`) and **Azure App Service** (script `04-deploy-app.sh`) are the two supported hosting targets.
- **Bash scripts with the Azure CLI** (`scripts/azure/01` to `06`, `99`) create the test workload, plant waste, create the read-only identity, deploy the app, trigger a cost spike, attach a custom domain and clean up.
- **pytest** runs 27 automated tests; **GitHub Actions** runs them on every push.
- **Headless Chromium (Playwright)** was used to load every page in desktop and 400-pixel widths and to capture screenshots.

Table 3.3: Main Python dependencies (requirements.txt)

Package	Version	Role
fastapi	0.115.0	Web framework and request validation
uvicorn[standard]	0.30.6	ASGI server
sqlalchemy	2.0.35	ORM and database access
apscheduler	3.10.4	Periodic and manual synchronisation jobs
pydantic	2.9.2	Typed request and response models
python-dotenv	1.0.1	Reads settings from the .env file
python-multipart	0.0.32	File upload for FOCUS bills
cryptography	>= 42	Fernet encryption of stored cloud secrets
azure-identity	1.25.3	OAuth 2.0 token credentials for Azure
boto3	1.35.0	Optional AWS collector
pytest, httpx	8.3.3, 0.27.2	Automated tests and API test client

3.8 Cloud Computing Concepts Applied in the Project

CloudPulse sits on top of several standard cloud computing topics. This section presents them as twelve short modules. Each module states the concept in brief and then the exact place where CloudPulse uses it, so that the theory can be traced to the implementation. Table 3.4 at the end of the section summarises the mapping.

Module 1: Cloud service and deployment models

Concept. Cloud services are offered as Infrastructure as a Service (IaaS: virtual machines, disks and networks that the customer manages), Platform as a Service (PaaS: a managed run-time such as Azure App Service) and Software as a Service (SaaS: a finished application). Deployment models are public, private and hybrid cloud. Billing is pay-as-you-go: cost follows consumption.

Applied in CloudPulse. CloudPulse monitors IaaS resources (VMs, disks, public IPs, snapshots) that carry most avoidable cost, can itself be deployed as PaaS on App Service, and is delivered to users as a SaaS-style web application with one workspace per customer.

Module 2: Virtualisation and compute

Concept. A virtual machine is a slice of a physical host with a chosen size (vCPU and memory). Burstable B-series sizes suit light workloads. A VM has a power state: when it is shut down from inside the operating system it is *stopped* but the hardware stays reserved and compute is still billed; only *deallocated* releases the hardware and stops the compute charge.

Applied in CloudPulse. The detector in `azure_collector.find_stopped_vms` separates the two states, and `analytics.vm_verdict` classifies running VMs as idle, oversized or fine from CPU and network metrics. The fix commands it suggests are `az vm deallocate` and `az vm delete`.

Module 3: Cloud storage

Concept. Azure managed disks come in tiers (Standard HDD, Standard SSD and Premium SSD) and are billed per provisioned gigabyte whether or not a VM is using them. A snapshot is a point-in-time copy of a disk and is also billed by size. Detaching a VM does not delete its disks.

Applied in CloudPulse. Unattached disks (`diskState = Unattached`) and orphaned or old snapshots are two of the four waste rules in Table 5.1. `prices.py` converts size and tier to a monthly figure.

Module 4: Cloud networking

Concept. A public IP address is a separate billable resource that can be associated with a network interface, a load balancer or a NAT gateway. A Standard-SKU static address is charged by the hour even when it is attached to nothing, while a dynamic address with nothing attached has no address and costs nothing.

Applied in CloudPulse. `find_idle_public_ips` reports static addresses with no `ipConfiguration` and no NAT gateway, and deliberately ignores dynamic ones.

Module 5: Identity and access management

Concept. Access to cloud resources is controlled by an identity provider (Microsoft Entra ID), identities (users, service principals, managed identities) and role-based access control

(RBAC) that grants roles at a scope. The principle of least privilege says an identity should have only the roles it needs. OAuth 2.0 access tokens prove who is calling.

Applied in CloudPulse. CloudPulse uses a service principal with only Reader and Cost Management Reader on one subscription, or a managed identity with no secret (Section 4.3). Its own users are separated by role: administrator, workspace owner and viewer.

Module 6: Resource management and governance

Concept. Azure organises resources in tenants, subscriptions and resource groups under Azure Resource Manager (ARM). Tags attach metadata such as owner or team, and Azure Policy can restrict regions or require tags. Governance means being able to say who owns what and what it is for.

Applied in CloudPulse. Resource Graph queries the whole inventory in one call; owner tags are a fallback for ownership attribution; untagged spend becomes a governance finding; the scripts read the subscription's allowed-location policy.

Module 7: Monitoring and observability

Concept. Azure Monitor collects platform metrics (for a VM: Percentage CPU, Network In Total and Network Out Total) with hourly or finer granularity. The Activity Log records control-plane operations, who performed them and when, for 90 days. Azure Advisor turns telemetry into recommendations in five categories, one of which is cost.

Applied in CloudPulse. Metrics feed the VM verdict, the Activity Log feeds the creator lookup in `owners.py` and `fetch_creator`, and Advisor items are merged into findings as supporting evidence.

Module 8: Cloud economics and pricing models

Concept. Compute can be bought pay-as-you-go, with one- or three-year reservations or savings plans at a discount, or as interruptible Spot capacity. Retail prices are published in a public price list. Usage is billed after it happens and Azure publishes cost data with a delay of several hours.

Applied in CloudPulse. The price book reads the Retail Prices API with a cache and fallback tables, and cost data is back-filled for 90 days. The 8 to 24 hour delay is why verification waits for three billed days after a fix.

Module 9: FinOps

Concept. FinOps is the practice of managing cloud cost through shared accountability. Its cycle is to inform (visibility and allocation), optimise (remove waste, rightsize, commit)

and operate (govern continuously) [2], [20]. The Cost Optimization pillar of the Azure Well-Architected Framework expresses the same ideas.

Applied in CloudPulse. The whole cost-to-action loop is an implementation of that cycle: the Overview and Cost explorer inform, findings and priority optimise, and the lifecycle with verification operates.

Module 10: Elasticity, scaling and cost

Concept. Elasticity means capacity can be added or removed quickly, which is why a short mistake, such as leaving a larger size running, appears on the bill within a day. Autoscaling and scheduled shutdown are the usual controls.

Applied in CloudPulse. `05-spike.sh up` scales the test workload for a day to create a real cost spike that the anomaly detector must flag, and `down` reverses it.

Module 11: Security and the shared responsibility model

Concept. The provider secures the infrastructure; the customer secures its identities, data and configuration. Practical controls include encryption at rest, secret management, hashed passwords, rate limiting and read-only access.

Applied in CloudPulse. Stored cloud secrets use Fernet encryption, passwords use scrypt, sessions are server-side, and the collector issues only read requests (Table 4.4).

Module 12: Automation, containers and DevOps

Concept. Cloud resources are created repeatably with the command-line interface or infrastructure-as-code tools (ARM templates, Bicep, Terraform). Containers package an application with its dependencies, and continuous integration runs tests on every change.

Applied in CloudPulse. The Azure scripts use the CLI, the Dockerfile and `docker-compose.yml` package the application, `render.yaml` and `04-deploy-app.sh` deploy it, and GitHub Actions runs the 27 tests.

Table 3.4: Cloud computing modules and where CloudPulse applies them

Module	Concept	Where in CloudPulse
1	Service and deployment models	IaaS monitored; PaaS and SaaS delivery
2	Virtualisation and compute	stopped vs deallocated VMs; idle and oversized verdict
3	Cloud storage	unattached disks; orphaned snapshots
4	Cloud networking	idle static public IPs
5	Identity and access management	service principal, RBAC, managed identity, token flow
6	Resource management and governance	Resource Graph, tags, untagged-spend finding
7	Monitoring and observability	Azure Monitor, Activity Log, Advisor
8	Cloud economics and pricing	price book, billing delay, 3-day verification wait
9	FinOps	inform, optimise, operate = the cost-to-action loop
10	Elasticity and scaling	05-spike.sh anomaly test
11	Security and shared responsibility	Fernet, scrypt, read-only collector
12	Automation, containers, DevOps	Azure CLI scripts, Docker, Render, App Service, CI

4. ARCHITECTURE DIAGRAM

4.1 Overview of the Architecture

CloudPulse is a three-tier web application that talks to a customer's Azure subscription. The **client tier** is a browser running a single-page application. The **server tier** is one Python process that contains the API, the security layer, the scheduler and all business logic. The **data tier** is a SQL database together with the server's configuration. The cloud itself is an external system: CloudPulse only ever reads from it, after proving its identity to Microsoft Entra ID. Section 4.2 describes the layers; Figure 4.1, on the full page that follows it, shows all components and the numbered flows between them, with the Azure credential path highlighted. The numbers are explained in Table 4.1.

4.2 Layers of the System

4.2.1 Client tier

The browser application has no build step. It calls the JSON API, keeps its state in memory, and renders ten pages: Overview, Cost explorer, Anomalies, Budgets, Findings, Owners, Savings, Data sources and, for administrators, Workspaces & users and the Audit log. A header on every state-changing request (`X-CloudPulse: 1`) lets the server reject forged cross-site form posts. Four kinds of people use it: the administrator who sees every workspace, a customer owner who can sync and set budgets, a customer viewer who can only look, and an anonymous demo visitor who sees sample data in read-only mode.

4.2.2 API and security layer

Three FastAPI routers expose the system: `/api/auth` (login, demo, logout, current user, password change), `/api/admin` (workspaces, Azure connections, users, memberships, audit) and `/api/*` (overview, explorer, month forecast, findings and their actions, savings, owners, notifications, budget, FOCUS upload, connections). Every request is authenticated by a server-side session, authorised against the user's role in the requested workspace, and logged where it changes data. Failed logins are limited to 8 attempts per 15 minutes per e-mail and per client address.

4.2.3 Service layer

The **collection orchestrator** coordinates one synchronisation. The **Azure collector** wraps the custom ARM client and converts API answers into *signals*. **Detectors and analytics** are pure functions without database or network access, which makes them easy to test. The **findings engine** turns signals into findings with a fingerprint, a priority and a lifecycle, and

runs the verification pass. The **owner resolver** finds the creator of each resource and caches the answer; the **price book** looks up Azure retail prices and caches them for seven days with built-in fallback tables for when the public API cannot be reached.

4.2.4 Data tier

All state is in one relational schema (Table 4.3). SQLite is the default because it needs no installation; setting `DATABASE_URL` to a PostgreSQL address moves the same schema to a production database.

4.3 Azure Credential Architecture

Everything CloudPulse knows about an Azure subscription comes through one identity. This section explains the Azure concepts involved and how the application keeps that identity safe.

4.3.1 Concepts

- **Tenant.** A Microsoft Entra ID directory that represents an organisation. Its identifier (tenant ID) is part of the token URL [11].
- **Subscription.** The billing and resource container that CloudPulse monitors. Its subscription ID appears in every ARM request path.
- **App registration and service principal.** An app registration defines an application identity (the **client ID**, also called the application ID). The service principal is its instance inside the tenant, the object to which roles are assigned.
- **Client secret.** A password generated for the app registration. CloudPulse needs it only to prove that the request really comes from the registered application. Secrets expire; the script issues one valid for one year.
- **Role-based access control (RBAC).** Permissions are granted as role assignments at a scope. CloudPulse is given **Reader** (list resources, read metrics and the Activity Log) and **Cost Management Reader** (read costs) on `/subscriptions/{id}` only [18]. Neither role can create, change or delete anything.
- **Access token.** The result of a successful sign-in: a signed JSON Web Token valid for about an hour. Every ARM request carries it in the `Authorization: Bearer` header; ARM checks the signature and the role assignments before answering.

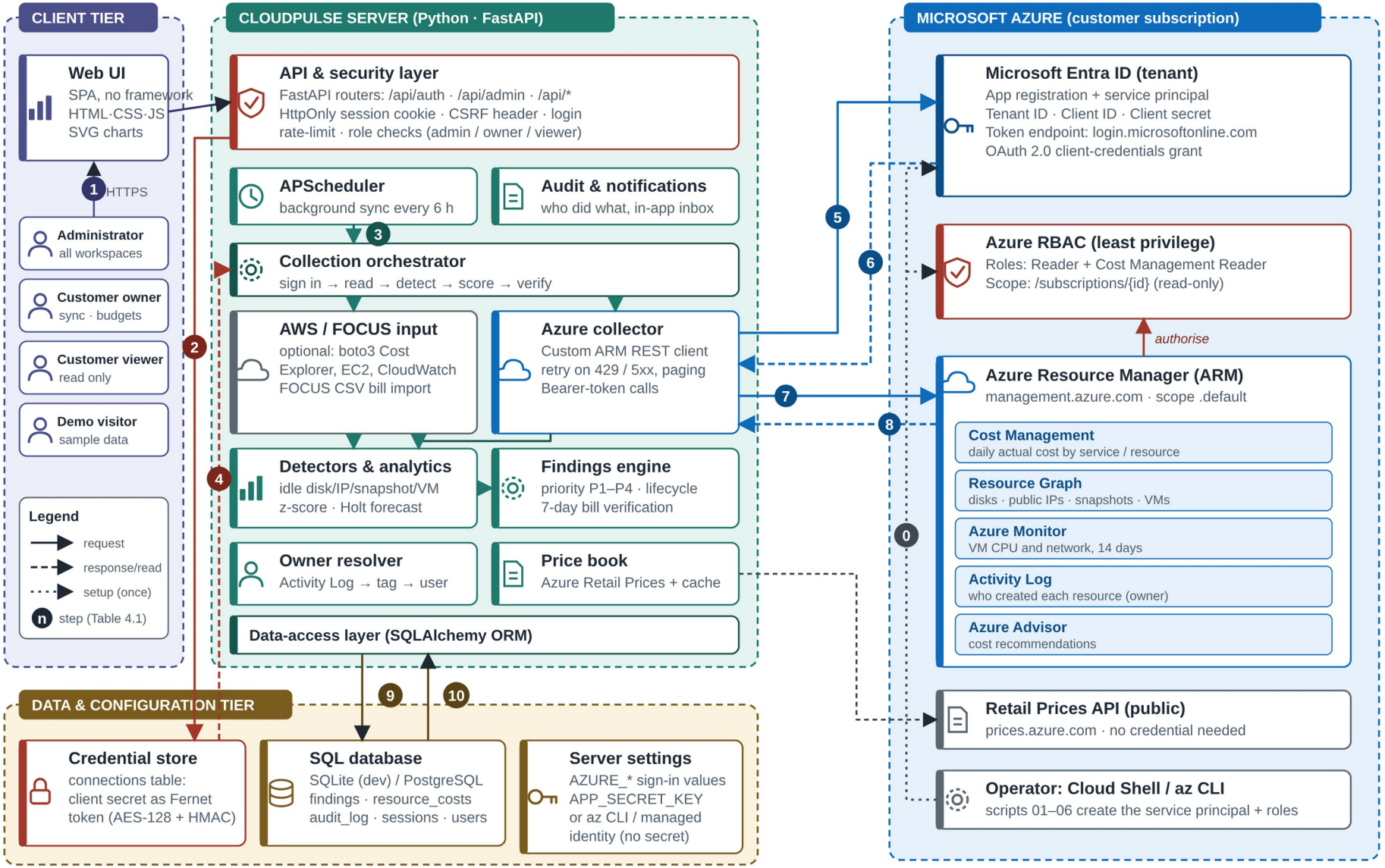


Figure 4.1: CloudPulse system architecture with the Azure credential and data flows

Table 4.1 explains every numbered flow of Figure 4.1; the credential-related steps (0, 2, 4, 5, 6, 7 and 8) are detailed in Section 4.3.2.

Table 4.1: Numbered flows in Figure 4.1

Step	Flow	What happens
0	Operator → Entra ID, RBAC	One-time setup in Azure Cloud Shell: an app registration and service principal are created and the roles Reader and Cost Management Reader are granted at subscription scope
1	Browser → API	A user signs in over HTTPS; the server checks the scrypt hash and sets an HttpOnly session cookie
2	API → credential store	The administrator submits subscription, tenant, client ID and client secret; the secret is encrypted with Fernet before it is stored
3	Scheduler → orchestrator	Every 6 hours, or when Sync is pressed, a collection run starts for each workspace
4	Credential store → collector	The connection is read and the secret decrypted in memory (or the server settings are used)
5	Collector → Entra ID	OAuth 2.0 client-credentials request for the scope management.azure.com/.default
6	Entra ID → collector	A short-lived JWT access token (about one hour) is returned and cached
7	Collector → ARM	Read-only REST calls with the Bearer token: cost, inventory, metrics, Activity Log, Advisor
8	ARM → collector	ARM validates the token and the role assignment, then returns JSON, or 403 / 404 / 429
9	ORM → database	Costs, findings, owners and events are written in a transaction
10	Database → ORM → API	The interface reads findings, charts and forecasts through the same API

The credential concepts above lead to three practical ways of signing in, compared in Table 4.2.

Table 4.2: Three ways CloudPulse can sign in to Azure

Method	Settings	Where the secret lives	Typical use
Service principal	AZURE_SUBSCRIPTION_ID, AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET (or the Connect Azure form)	Server settings, or encrypted in the connections table	CloudPulse on a laptop or any host outside Azure
Azure CLI login	AZURE_USE_CLI=true after `az login`	Azure CLI token cache on the same machine	Quick local demonstration
Managed identity	AZURE_USE_MANAGED_IDENTITY=true	None: Azure issues the token to the App Service	CloudPulse deployed on Azure App Service

4.3.2 Sign-in and request sequence

Figure 4.2 shows the complete sequence, from the one-time creation of the identity to a single read call. In phase B the `ClientSecretCredential` class posts the client credentials to the tenant's token endpoint and caches the returned token. In phase C each

collector call attaches the token; if the answer is HTTP 403 the message "Signed in, but this identity has no Reader role on the subscription" is shown to the administrator, and if it is HTTP 429 the client waits for the `Retry-After` period and tries again, up to a fixed number of attempts.

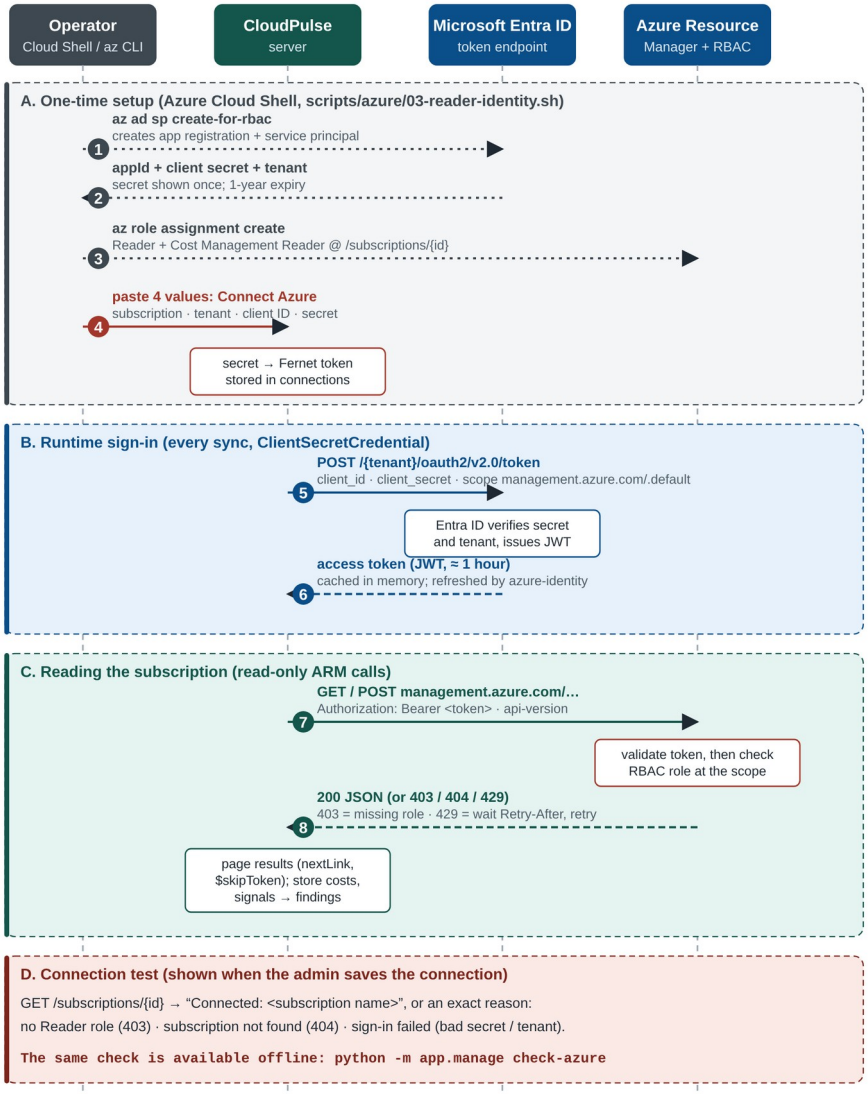


Figure 4.2: Azure credential creation, token acquisition and read-only request sequence

4.3.3 Protecting the credentials

- **Least privilege.** The identity is read-only and limited to one subscription; leaking it would expose information but not allow any change.
- **Encryption at rest.** A client secret entered in the interface is encrypted with **Fernet** (AES-128-CBC with an HMAC-SHA256 integrity check) using a key derived by SHA-256 from the server's `APP_SECRET_KEY`. Only the token is stored; the API never returns it, and the interface shows only whether it can still be read.

- **No secret at all on Azure.** When deployed on App Service the managed identity [19] needs no stored credential.
- **Separation of settings.** Credentials in `.env` are excluded from version control by `.gitignore`; the README warns never to place secrets in chats, screenshots or git.
- **Rotation.** Because the secret expires after a year, the setup script can be run again to issue a new one, and the old connection can be replaced in the administrator page.
- **Early diagnosis.** The command `python -m app.manage check-azure` signs in with the configured settings and reports either "Connected" with the subscription name or the exact reason for failure.

4.4 Data Model

Table 4.3: Database tables

Table	Purpose	Key columns
users, memberships	Accounts and their role (owner / viewer) in each workspace	email, password_hash (scrypt), is_admin, must_change_password
sessions	Server-side login sessions, 12-hour lifetime	token hash, expiry
workspaces	One organisation's cloud data; kind is customer or demo	slug, name, kind
connections	Cloud accounts of a workspace	provider, auth (env / service_principal / cli / managed_identity), config JSON, secret (Fernet)
cost_snapshots	Daily cost by service and provider	day, service, provider, amount
resource_costs	Daily cost of single resources (used for verification)	resource_id, day, amount
resource_owners	Cached creator of each resource	principal, source, checked_at
findings	One row per problem, unique by fingerprint	kind, monthly_impact, priority_score, status, assignee_id, realized_monthly, evidence JSON
finding_events	History of a finding	kind, message, actor, at
notifications	In-app inbox	user_id, message, read
audit_log	Sign-ins and administrative changes	actor, action, detail, at
price_cache, settings, collection_runs	Retail price cache, workspace budget, sync history	sku, price, step, status

4.5 Deployment Architecture

The same container image runs in four settings. On a laptop the application starts with `uvicorn` and a SQLite file. With Docker Compose a named volume keeps the database between restarts. On Render the blueprint `render.yaml` builds the Dockerfile, generates a random `APP_SECRET_KEY`, and exposes a health check at `/api/health`. On Azure App Service the script `04-deploy-app.sh` creates the web app, enables its managed identity, grants it the two reader roles and asks for the administrator's e-mail and password,

so that no cloud secret is stored anywhere. A custom domain with a free HTTPS certificate can be attached with `06-custom-domain.sh`.

4.6 Security Architecture

Table 4.4: Threats and the controls that address them

Threat	Control in CloudPulse
Stolen password database	scrypt (N = 2^14, r = 8, p = 1) with a random salt per user; temporary passwords must be changed at first sign-in
Password guessing	Rate limit of 8 failures per 15 minutes; generic error text
Session theft	Random token, only its SHA-256 hash is stored; HttpOnly, SameSite=Lax and (on HTTPS) Secure cookie; 12-hour lifetime
Cross-site request forgery	Every non-GET request must carry the custom `X-CloudPulse` header, which a foreign site cannot set
Leaked cloud credential	Read-only roles on one subscription; Fernet encryption at rest; managed identity option with no secret
Cross-tenant access	Every query is scoped by workspace and checked against the user's membership
Clickjacking and sniffing	X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy same-origin
Abuse of the demo	Demo user is read-only and bound to the sample workspace only
Accidental change to the cloud	The collector issues only GET requests and the Cost Management and Resource Graph query POSTs

5. METHODOLOGY

5.1 Development Approach

The project followed an incremental, test-first approach. The algorithms were written first as pure Python functions with unit tests whose expected values were calculated by hand, then the Azure collector was added and tested against a fake cloud, then the findings engine and API, and finally the user interface. A scripted Azure workload was created so that the same code could be proven on a real subscription. Each stage kept the previous tests passing; the final suite contains 27 tests.

5.2 The Cost-to-Action Loop

The loop of Chapter 1 maps one-to-one to software modules: signals come from `azure_collector.py` and `workspace_detectors.py`; context from `prices.py` and the cost tables; findings and priority from `findings.py` and `analytics.py`; ownership from `owners.py`; action from the routers and `findings.act`; verification from `findings.verify`. The central data structure is the **signal**, a plain dictionary with a kind, a resource, a monthly impact, a confidence, evidence and suggested fix commands. A signal that survives de-duplication becomes a **finding**.

5.3 Data Collection

A synchronisation is one function call, `sync_workspace`, executed by APScheduler or by the Sync button (Figure 5.1). It is deliberately fault tolerant: every collection step records its own error, so a permission problem with Azure Advisor does not stop the cost data from loading, and the Data sources page shows which step failed and why.

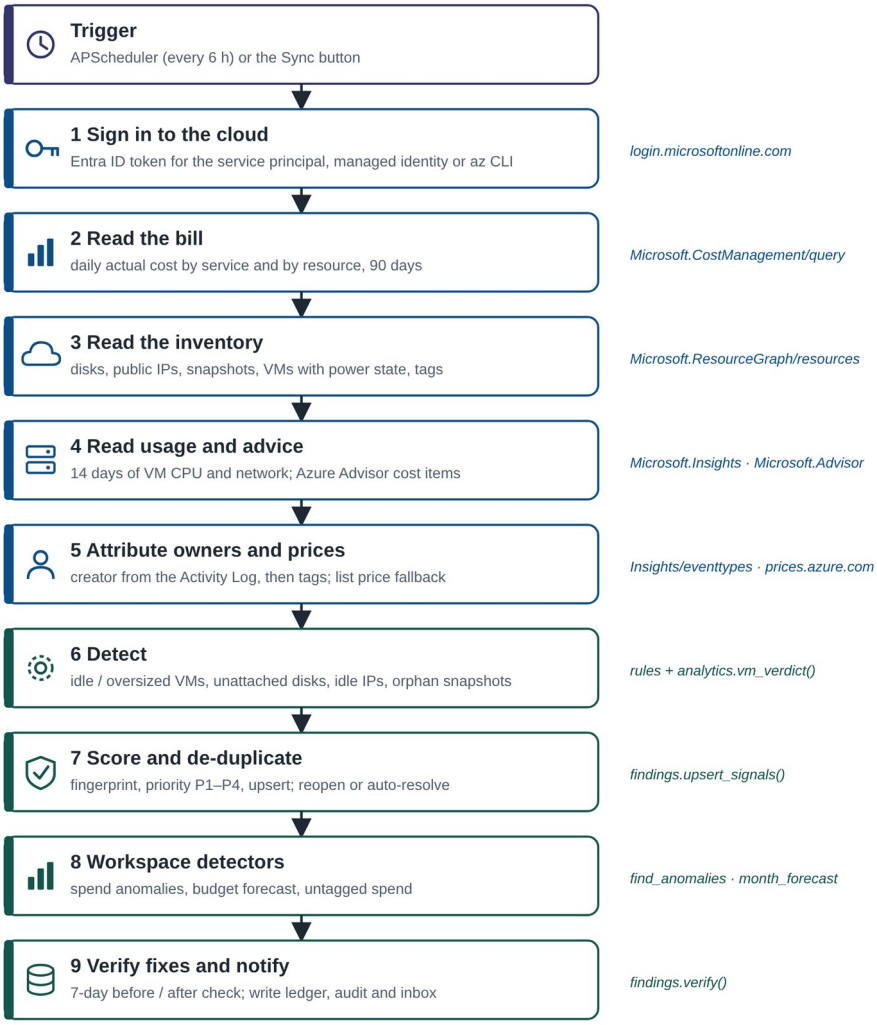


Figure 5.1: The synchronisation pipeline executed on every sync

Cost data. The Cost Management query API is called with an `ActualCost` definition, daily granularity and grouping by service, and again by resource. The first sync back-fills 90 days; later syncs re-read only the last 10 days because Azure revises recent days. **Inventory.** A single Resource Graph query lists every resource, and a second returns disks, public IPs, snapshots and VMs with the properties needed for detection. Resource Graph answers are paged with `$skipToken`. **Usage.** For each running VM, hourly CPU and network metrics for the previous 14 days are requested from Azure Monitor. **Advice.** Azure Advisor cost recommendations are read and merged into matching findings as supporting evidence.

5.4 Detection Rules

Table 5.1: Waste detection rules applied to the Azure inventory

Finding	Rule	Monthly impact	Confidence
Unattached disk	`diskState = Unattached`	Size × list price of the disk tier (retail price API, fallback table)	0.95
Idle public IP	Static allocation, no `ipConfiguration`, no NAT gateway	Standard public IP list price	0.95
Orphaned snapshot	Source disk no longer exists; or older than 30 days	Size × snapshot price	0.90 / 0.60
VM stopped but billed	Power state `stopped` (not deallocated)	Hours per month × VM list price + attached disks	0.90
Deallocated VM	Power state `deallocated`; disks still billed	Attached disks only	0.60
Idle VM	p95 CPU < 5 % and p95 network < 5 MB per hour over the window	Compute price of the size	0.85 (0.65 if < 7 days)
Oversized VM	p95 CPU < 20 % and peak CPU < 80 %; smaller size on the ladder exists	Price difference to the next smaller size	0.70 (0.50)
Advisor item	Azure Advisor, category Cost	Annual saving ÷ 12 as published by Advisor	0.80 / 0.70 / 0.60 by Advisor impact

Where Cost Management already reports the billed cost of a resource over the last days, the **actual cost** replaces the list-price estimate, because enterprise agreements and reservations make list prices unreliable. The finding records which basis was used ("list price" or "billed cost"). For VMs, memory cannot be measured without a guest agent, so the oversized verdict is always 0.15 less confident than the idle verdict.

5.5 Ownership Attribution

For every resource that produced a signal, CloudPulse queries the Activity Log (management events of the last 89 days) filtered to that resource. It selects the **earliest successful `/write` operation**, whose `caller` field is either a user principal name, or a GUID for an application or managed identity. A GUID caller is described as a "service identity" together with its application ID. If the Activity Log has no event, the tags `owner`, `createdby`, `created-by`, `created_by`, `contact` and `team` are tried in that order. The result is cached in `resource_owners`; known owners are kept, unknown ones are retried once a day. A person matched by e-mail to a CloudPulse user receives the finding as an assignment and a notification.

5.6 Anomaly Detection

A good anomaly detector must ignore normal weekly rhythm, such as lower spend at weekends, and must not be fooled by earlier anomalies. CloudPulse therefore uses a

robust, weekday-aware z-score based on the median and the median absolute deviation (MAD), with the cut-off of 3.5 recommended for modified z-scores [8]. For each billed day *x*:

```
B = same weekday, previous 4 weeks (>= 3 of them), else previous 14 days
m = median(B)
s = max(1.4826 x MAD(B), 0.05 x m, 0.01)     # floor: a flat history cannot
give s = 0
z = (x - m) / s
flag the day when  z > 3.5  and  (x - m) >= max(1.00, 0.20 x m)
```

At least seven billed days are required. The **drivers** of a flagged day are found by comparing each service with its own median on the baseline days; the increases are expressed as shares that add up to 100 %. A spike is **ongoing** if the next two days both stay above *m* plus half the excess; the impact is then projected as excess × 30.4 days, otherwise it is the one-off excess. Confidence grows with the z-score: min(0.95, 0.55 + 0.05 × (z − 3.5)). **Worked example (demo data, Tuesday 24 September 2024):** the baseline was the three earlier Tuesdays (3, 10 and 17 September) with a median of $73.55 and a robust spread of $12.11. The day cost $169.26, an excess of $95.71 (2.3 times the usual), giving z = 95.71 / 12.11 = 7.9, far above 3.5. Amazon Elastic Compute Cloud explains 99.1 % of the increase (from a usual $19.04 to $164.14). Both following days stayed high, so the spike is *ongoing* and its monthly impact is 95.71 × 30.4 = $2,909.55, with confidence 0.55 + 0.05 × (7.9 − 3.5) = 0.77. The day is shown in Figure 6.4.

5.7 Forecast and Budget Breach

Month-end spend is forecast with **Holt's linear exponential smoothing** [9], [10], a method that tracks a level and a trend. Before fitting, each day is divided by a **weekday factor** (the weekday's average over the overall average, clipped to the range 0.5 to 1.5), which needs four weeks of data. Smoothing constants are chosen by trying every pair from $\alpha \in \{0.2, 0.4, 0.6, 0.8\}$ and $\beta \in \{0.05, 0.1, 0.2\}$ and keeping the pair with the smallest one-step-ahead squared error. Each remaining day *h* is projected as

```
forecast(h) = max(0, level + h x trend) x weekday_factor(day)
month_end   = month_to_date + sum(forecast(h))
80% range   = month_end +/- 1.2816 x RMSE x sqrt(days_left) x mean(factors)
breach date = first day on which the running total passes the budget
```

With fewer than seven billed days the average of the days so far is used, and once the month is complete the forecast equals the actual total. A **budget risk** finding is raised when the forecast passes the budget. In the demo data the $2,600 budget was passed on 28 September, ending the month at $2,829.99, 9 % over.

5.8 Priority Score

Findings differ in money, urgency and certainty, so CloudPulse combines them into one number from 0 to 100:

```
score = 100 x (0.6 x impact_score + 0.4 x severity_weight)
              x confidence x age_factor
impact_score    = min(1, log10(1 + impact) / log10(1 + 1000))
                  # $10 -> 0.35, $100 -> 0.67, $1,000 -> 1.0
severity_weight = critical 1.0 | high 0.8 | medium 0.55 | low 0.3
                  # severity from impact: >= 500, >= 100, >= 20, else low
age_factor      = 1 + min(0.3, days_open / 100)     # up to +30 %
band            = P1 >= 70 | P2 >= 45 | P3 >= 25 | P4 otherwise
```

The logarithm prevents one very large item from hiding all others, and the age factor gradually raises forgotten findings. Governance findings such as untagged spend count only 25 % of their dollar value, because that spend is not wasted. **Worked example:** a stopped VM costing $70.08 a month with confidence 0.9 has impact_score log10(71.08)/log10(1001) = 0.617 and medium severity (0.55), so the score is 100 × (0.6 × 0.617 + 0.4 × 0.55) × 0.9 = 53 (P2). After 30 days open the age factor is 1.3 and the score becomes 69, still P2. Figure 5.3 plots the function for the four severities.

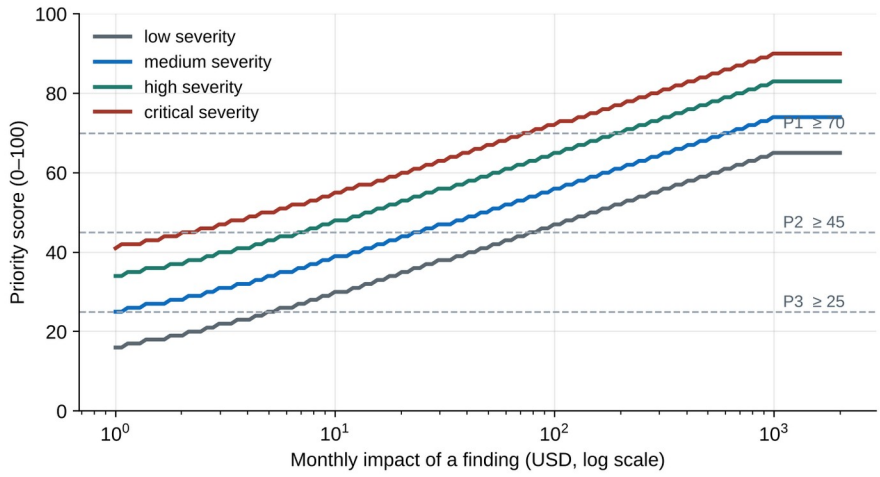


Figure 5.3: Priority score against monthly impact (confidence 0.9, new finding), produced by the implemented function

5.9 Finding Lifecycle

A finding moves through the states of Figure 5.2. Permissions are role based: administrators and workspace owners may perform every action, the assignee may start and resolve their own finding, viewers and the demo user may only read. Dismissing requires a written reason of at least three characters and snoozing is limited to 1 to 90 days. Each action writes a `finding_event`, updates the score, and notifies the assignee. A finding is identified by a **fingerprint**, a SHA-256 hash of its kind and resource, so that the same problem found on consecutive syncs updates one row instead of creating many.

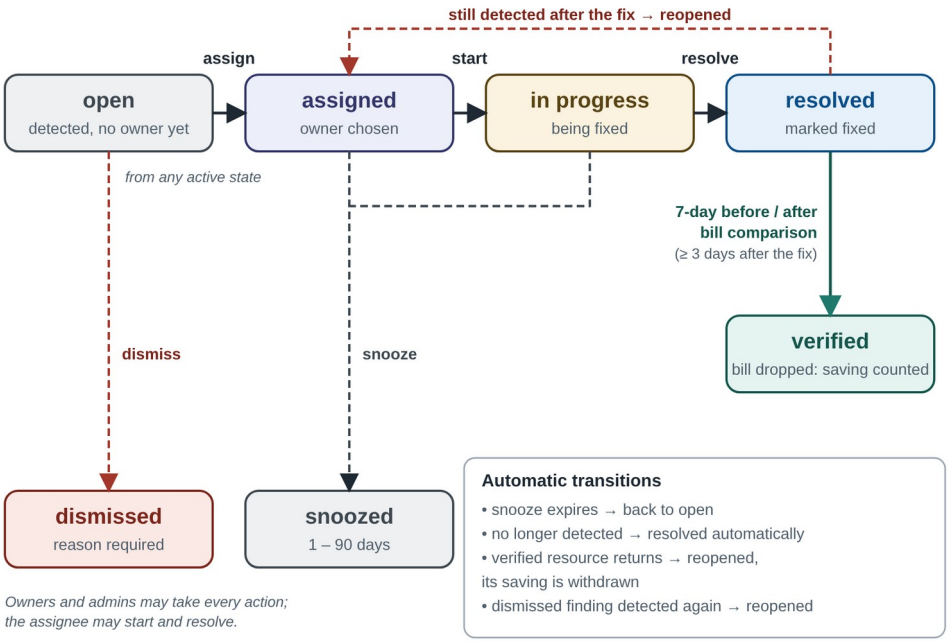


Figure 5.2: Finding lifecycle, with automatic and manual transitions

5.10 Saving Verification

Verification is what distinguishes CloudPulse from a report generator. When a finding is marked resolved, the next sync checks that the resource is no longer detected (a 30-minute grace period tolerates delay in Azure). Then the billed cost is compared:

```
before = mean daily cost of the resource, 7 days before the fix day
after  = mean daily cost, up to 7 days after the fix (billed days only)
realised_monthly = max(0, before - after) x 30.4
verify only when at least 3 'after' days exist
```

If the cost has not dropped yet the finding waits, up to 14 days; a resource with no billed history is verified with its estimate and the method is recorded. A verified saving shows the **estimate accuracy**, the realised saving divided by the estimate. Anomalies are verified when the last three billed days are back within half the spike. If a verified resource appears again, the finding is reopened and the saving is withdrawn.

5.11 Notifications, Audit and Export

Assignments, comments, resolutions, dismissals and verifications create notifications that appear under the bell icon with an unread counter. Sign-ins, failed sign-ins, user changes, connection changes and syncs are written to the audit log. The findings list can be searched, filtered by kind, priority and assignee, and exported to CSV.

5.12 The Azure Test Environment

To demonstrate CloudPulse on a real subscription, a set of Bash scripts creates a small company and then plants waste. They read the subscription's allowed regions, register the needed resource providers, and ask for confirmation before changing anything.

Table 5.2: Azure scripts

Script	Action	Approx. cost
01-workload.sh	VM, App Service plan and web app, Azure SQL Basic database, storage account with sample data	$2 / day
02-leaks.sh	Plants four untagged wasted resources (Table 5.3)	$0.30 / day
03-reader-identity.sh	Creates the read-only service principal and prints the four values CloudPulse needs	free
04-deploy-app.sh	Deploys CloudPulse to App Service with a managed identity	about $13 / month
05-spike.sh up | down	Scales the workload for a day to create a real anomaly	about $7
06-custom-domain.sh	Attaches a custom domain with free HTTPS	free
99-cleanup.sh [--all]	Deletes the test resources (and the app and identity with --all)	-

Table 5.3: Waste planted by 02-leaks.sh and the finding CloudPulse raises

Planted resource	Azure command used	Finding raised
leftover-disk (4 GB)	az disk create (not attached)	Unattached disk
unused-ip	az network public-ip create (Standard, static)	Idle public IP
old-backup-snap	snapshot of temp-disk, then the disk is deleted	Orphaned snapshot
forgotten-vm	az vm create then az vm stop (not deallocate)	VM stopped but still billed

None of the planted resources has an owner tag; CloudPulse must find the creator from the Activity Log, which is exactly the capability being demonstrated. Because Azure publishes costs 8 to 24 hours late, the scripts are run a day before a demonstration; waste and owners appear within minutes, while cost charts follow later.

5.13 Testing Strategy

Table 5.4: Automated tests (27)

File	Tests	What is verified
tests/test_analytics.py	23	median, percentile and MAD; flat series raises no anomaly; spike detection with weekday baseline and drivers; ongoing spike projection; small increases ignored; 7-day minimum; 14-day fallback; forecast for complete, constant, linear and short series; priority values and bands; governance down-weighting; severity tiers; VM idle / oversized / fine; size ladder; before/after for deleted, waiting and partial drops; saved-to-date
tests/test_engine.py	1	Full loop with a fake Azure cloud: sync, findings, owners, assign, start, dismiss, resolve, reopen, verify, ledger, permissions (a viewer may resolve their own finding but not dismiss), regression when a fixed resource returns, audit trail
tests/test_static.py	3	CSS colour tokens are valid and opaque; every page has a renderer; Azure setup diagnostics explain what is missing

Beyond unit tests, every page was loaded in headless Chromium at desktop width and at 400 pixels, with the browser console checked for errors. This check found a real defect: a mistyped colour value in the style sheet made the main text almost transparent. It was corrected and a regression test was added.

6. RESULTS

This chapter presents the running system. The five screenshots were taken from the application served at `127.0.0.1:8000` in a desktop browser with the dark theme, which the interface selects automatically from the operating-system preference. They show the sign-in page and the four main analysis pages. All figures that appear in the screenshots come from the application's built-in demonstration workspace, which runs the same collection, detection, scoring and verification code as a live Azure workspace on the bundled FOCUS sample bill.

6.1 Experimental Set-up

Table 6.1: Set-up used for the results in this chapter

Item	Value
Application	CloudPulse v5 served by Uvicorn on 127.0.0.1:8000, SQLite database
Data	FOCUS sample bill (100,000 records) aggregated to 1,166 daily rows, 21 March to 30 September 2024; 12 example findings with scripted owners and history
Clock	The demonstration fixes its "today" at 1 October 2024 so that the figures are reproducible
Accounts	Administrator (Overview screenshot) and read-only demo visitor (the other pages)
Browser	Desktop Chrome, 1920 × 1080; interface verified in headless Chromium at 400 px as well
Tests	pytest, 27 passed

6.2 Sign-in Page

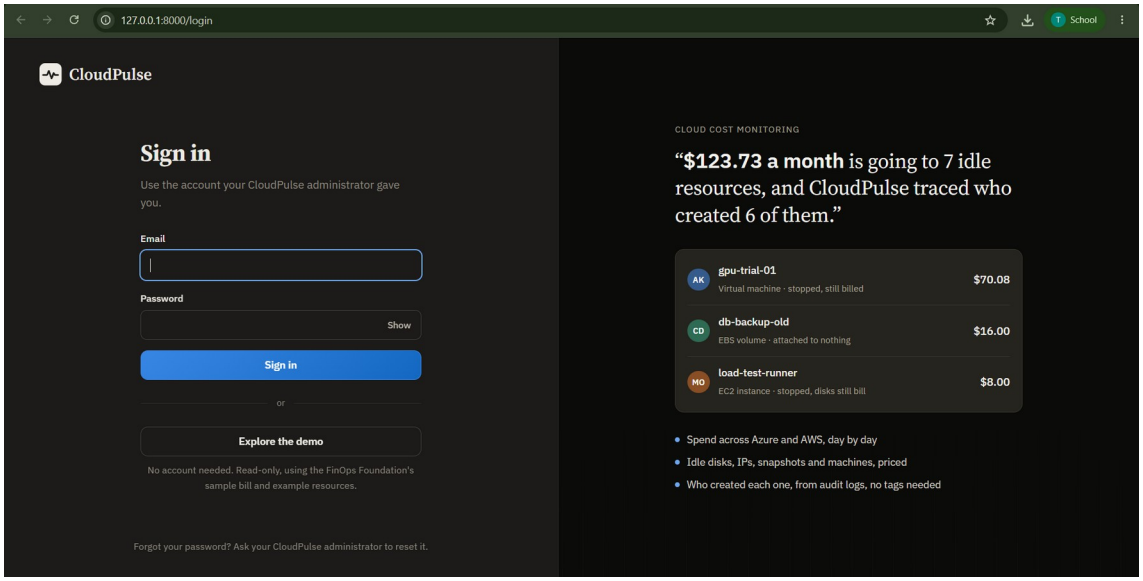


Figure 6.1: CloudPulse sign-in page (127.0.0.1:8000/login)

Figure 6.1 is the entry point of the system. The left half contains the **Email** and **Password** fields, a **Show** switch for the password, the blue **Sign in** button and a second, outlined button, **Explore the demo**. Signing in uses the account the CloudPulse administrator created; there is deliberately no self-registration, and the note at the bottom, "Forgot your password? Ask your CloudPulse administrator to reset it", reflects that the administrator resets passwords with a temporary one that must be changed at first use. **Explore the demo** opens the read-only sample workspace without any account, which is the mode used for faculty and visitor demonstrations and for all other screenshots; the grey note under the button states that it uses the FinOps Foundation's sample bill and example resources.

The right half states what the product does with live examples computed by the application: "**$123.73 a month** is going to **7 idle resources**, and CloudPulse traced who created **6 of them**." Three rows illustrate it. **gpu-trial-01**, a virtual machine that is *stopped, still billed*, costs $70.08 a month and is attributed to the person with initials AK; **db-backup-old**, an EBS volume *attached to nothing*, costs $16.00; **load-test-runner**, an EC2 instance that is *stopped* while its *disks still bill*, costs $8.00. The coloured circles are the creators found from audit logs. The three bullets summarise the capabilities delivered in this report: spend across Azure and AWS day by day, idle resources priced, and the creator of each one found from audit logs without needing tags. This page therefore demonstrates the authentication feature (FR1) and the product's value proposition in one view.

6.3 Overview Page

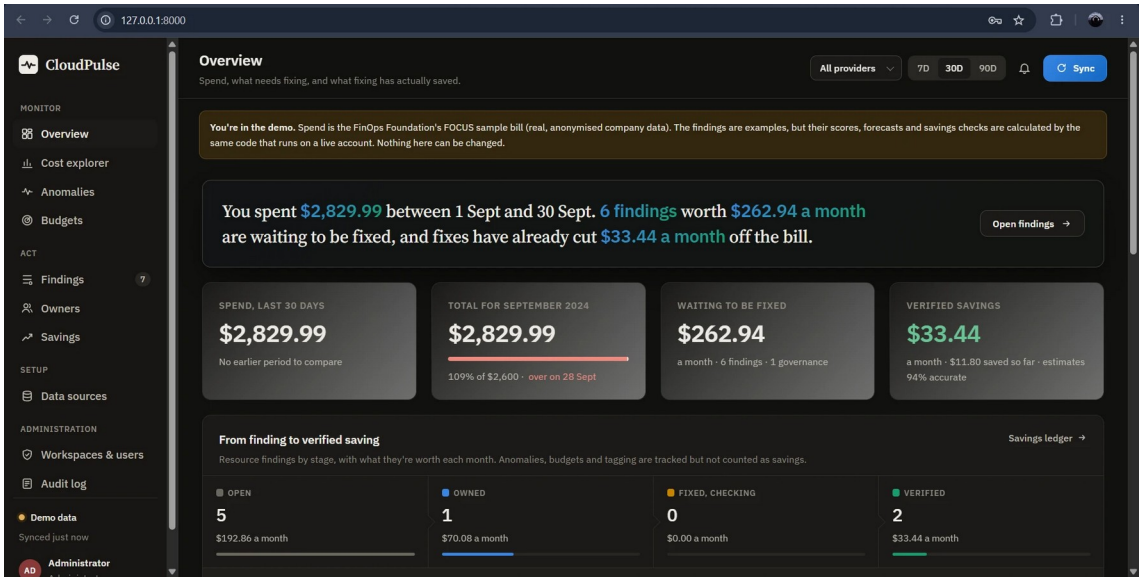


Figure 6.2: Overview page (administrator view of the demo workspace)

The Overview page, Figure 6.2, is the first page after sign-in and answers three questions at once: what was spent, what needs fixing, and what has fixing already saved. The **left navigation** is grouped into *Monitor* (Overview, Cost explorer, Anomalies, Budgets), *Act* (Findings with a badge of **7** items still to do, Owners, Savings), *Setup* (Data sources) and, because the screenshot was taken as administrator, *Administration* (Workspaces & users, Audit log). The footer of the sidebar shows the data source ("Demo data") and when it last synchronised, and the signed-in user. In the top bar are the provider filter (**All providers**), the range selector **7D / 30D / 90D**, the notification bell and the blue **Sync** button, which is visible only to users who may start a collection. The amber banner reminds the viewer that the data is sample data.

The large sentence is generated from the numbers below it: "You spent **$2,829.99** between 1 Sept and 30 Sept. **6 findings** worth **$262.94 a month** are waiting to be fixed, and fixes have already cut **$33.44 a month** off the bill." The four cards repeat the figures with context. **Spend, last 30 days** is $2,829.99. **Total for September 2024** is also $2,829.99, drawn against the budget as a red bar, "109% of $2,600, over on 28 Sept". **Waiting to be fixed** is $262.94 a month made of 6 findings plus 1 governance finding. **Verified savings** is $33.44 a month, of which $11.80 has already been saved to date, and "estimates 94% accurate", meaning the realised saving divided by the original estimate.

The panel **From finding to verified saving** displays the whole loop as four stages with the monthly value of each: **Open** 5 findings ($192.86), **Owned** 1 ($70.08), **Fixed, checking** 0 ($0.00) and **Verified** 2 ($33.44). The open and owned values add up to the $262.94 shown above, which is a useful consistency check between cards. The strip below lists items that are tracked but not counted as savings: one dismissed with a reason, one snoozed, one anomaly being tracked and one governance finding. This single screen therefore evidences the signal-to-verification loop (FR5, FR8, FR9) and the budget forecast (FR7).

6.4 Cost Explorer

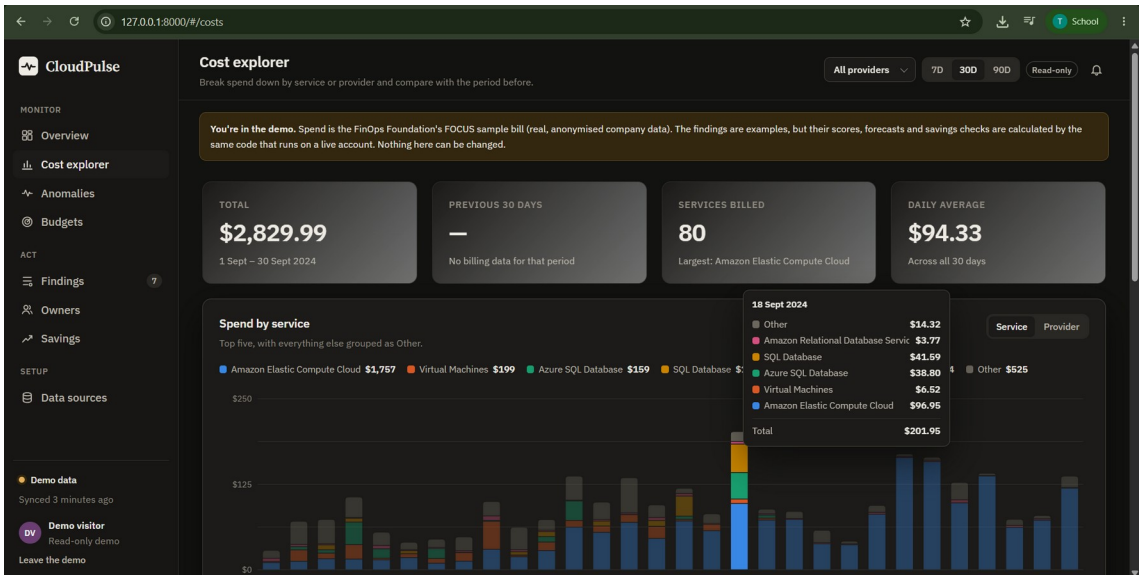


Figure 6.3: Cost explorer with the daily tooltip for 18 September 2024

The Cost explorer, Figure 6.3, breaks the bill down by service or provider. The four cards report **Total** $2,829.99 for 1 to 30 September 2024, **Previous 30 days** shown as a dash because the sample contains no billing data for that comparison period, **Services billed** 80 with the largest being *Amazon Elastic Compute Cloud*, and **Daily average** $94.33, which is the total divided by 30 days. The card values are consistent with the Overview.

The chart **Spend by service** draws one stacked bar per day with the top five services in distinct colours and everything else grouped as *Other*; the **Service / Provider** switch at the right changes the grouping. The legend shows the 30-day totals: Amazon Elastic Compute Cloud $1,757, Virtual Machines $199, Azure SQL Database $159, SQL Database and others, and Other $525. In this screenshot the pointer is on **18 September 2024**, and the tooltip lists that day's cost per service: Amazon Elastic Compute Cloud $96.95, Virtual Machines $6.52, Azure SQL Database $38.80, SQL Database $41.59, Amazon Relational Database Service $3.77 and Other $14.32, giving a **total of $201.95**. The sample bill mixes AWS and Azure services, which demonstrates the multi-provider normalisation done by the FOCUS format. This page covers FR3 and FR10: reading daily cost by service and importing bills in the FOCUS format.

6.5 Anomalies Page

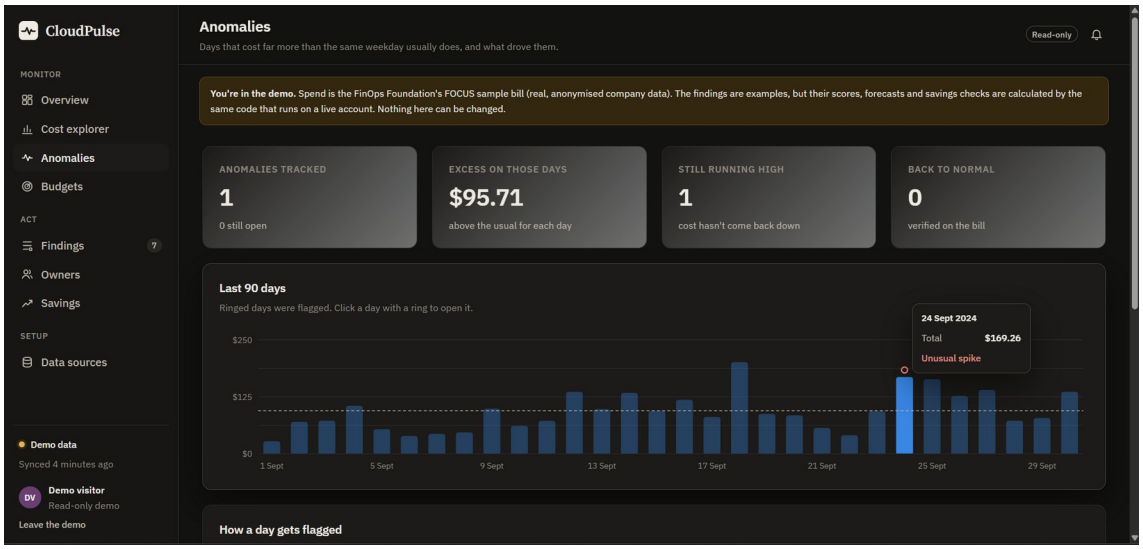


Figure 6.4: Anomalies page showing the flagged day 24 September 2024

Figure 6.4 shows the anomaly detector's output. The summary cards state that **1 anomaly** is tracked (0 still open, because its finding has already been marked as fixed), that the **excess on those days** is **$95.71** above the usual cost for each day, that **1** anomaly is **still running high** ("cost hasn't come back down") and that **0** are **back to normal, verified on the bill**. The last card is what keeps CloudPulse honest: marking an anomaly as fixed is not enough, the bill has to return to normal for three billed days.

The chart **Last 90 days** draws daily spend as bars with a dashed average line; "Ringed days were flagged. Click a day with a ring to open it." The ringed bar is **24 September 2024**, and the tooltip reads **Total $169.26**, **Unusual spike**. Following the method of Section 5.6, the same weekday of the previous weeks had a median of $73.55, so the excess is $95.71 or 2.3 times the usual cost, with a robust z-score of 7.9 (the threshold is 3.5). The next two days also stayed high, so the spike was classified as *ongoing* and its monthly impact projected as $2,909.55 in the findings list. Opening the day lists the services that caused the increase: Amazon Elastic Compute Cloud accounts for 99.1 % of it, rising from a usual $19.04 to $164.14. Below the chart, the section **How a day gets flagged** explains the rule in plain language. Other tall days are not ringed when they fail the rule against their own weekday baseline, for example 18 September ($201.95 in Figure 6.3), which keeps false alarms low. This page demonstrates FR7.

6.6 Budgets Page

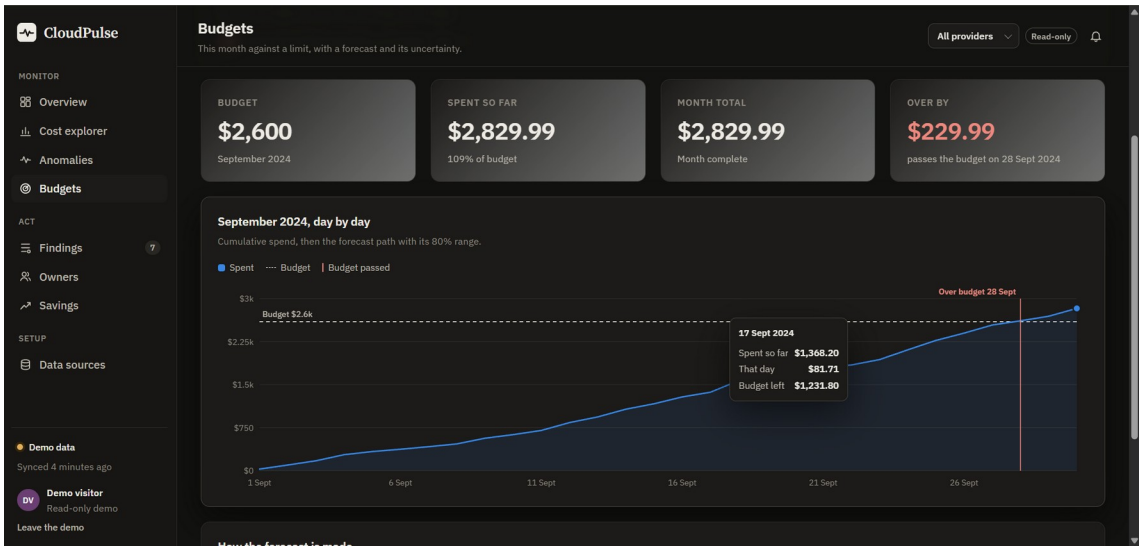


Figure 6.5: Budgets page with the cumulative spend line and the budget breach marker

The Budgets page, Figure 6.5, compares the month with a limit that the workspace owner sets. The four cards show the **Budget** $2,600 for September 2024, **Spent so far** $2,829.99 (109 % of budget), **Month total** $2,829.99 ("Month complete") and **Over by** $229.99 in red, with the note "passes the budget on 28 Sept 2024". The chart **September 2024, day by day** shows the cumulative spend as a blue line and area, the budget as a dashed horizontal line labelled "Budget $2.6k", and a vertical red marker labelled **Over budget 28 Sept** where the line crosses. The tooltip for **17 September 2024** reads "Spent so far $1,368.20; That day $81.71; Budget left $1,231.80", and $2,600 − $1,368.20 = $1,231.80 confirms the arithmetic.

Because the demonstration month is already complete, the forecast equals the actual total and the uncertainty band collapses to a line. For a month in progress the same chart extends the line with the Holt forecast and an 80 % shaded band, and the breach marker moves to the *predicted* day. The panel below the chart, **How the forecast is made**, documents the method for the reader. The corresponding **Budget risk** finding is the highest ranked item in the system with a score of 75 (P1), which links this page to the findings workflow.

6.7 Findings Produced and Their Priority

Table 6.2 lists the twelve findings of the demonstration workspace in order of priority score, as stored by the application. The ordering shows the scoring at work: the budget risk and the large idle virtual machines outrank small idle IP addresses, and a verified item keeps its original score.

Table 6.2: Findings in the demo workspace (impact in USD per month)

#	Finding	Kind	Cloud	Impact	Score	Status
1	September 2024 ended 9% over budget	Budget risk	-	230.00	75 (P1)	Open
2	VM gpu-trial-01 is stopped but still billed	Idle resource	Azure	70.08	69 (P2)	In progress
3	VM reporting-vm runs but does almost nothing	Idle VM	Azure	70.08	65 (P2)	Open
4	Spend on 24 Sep was 2.3x the usual	Spend anomaly	AWS	2,909.55*	65 (P2)	Resolved (checking)
5	VM api-prod-01 is larger than it needs to be	Oversized VM	Azure	70.08	54 (P2)	Open
6	Advisor: reserved instances to save over pay-as-you-go	Advisor	Azure	41.50	50 (P2)	Open
7	Disk web-01_OsDisk_old is attached to nothing	Idle resource	Azure	19.71	43 (P3)	Verified
8	EBS volume db-backup-old is attached to nothing	Idle resource	AWS	16.00	39 (P3)	Verified
9	EC2 instance load-test-runner is stopped; volumes still bill	Idle resource	AWS	8.00	36 (P3)	Open
10	Public IP staging-lb-ip points at nothing	Idle resource	Azure	3.65	31 (P3)	Dismissed
11	Elastic IP 13.232.81.40 is attached to nothing	Idle resource	AWS	3.60	31 (P3)	Snoozed
12	Snapshot pre-upgrade-snap is no longer needed	Idle resource	Azure	3.20	29 (P3)	Open

** An ongoing anomaly is projected as excess × 30.4 days; it is an investigation and is not counted as a saving.*

6.8 Savings Ledger

Only findings that reached the *verified* state contribute to savings. In the demonstration two resources were fixed and then confirmed by the before/after bill comparison (Table 6.3). The estimate for the two items was $35.71 a month and the realised saving $33.44 a month, an accuracy of 93.6 %, which the interface rounds to 94 %. The "saved to date" figure of $11.80 is the realised monthly saving multiplied by the days since each verification divided by 30.4.

Table 6.3: Verified savings in the demo workspace

Resource	Cloud	Estimated (USD / month)	Realised (USD / month)	Accuracy
Disk web-01_OsDisk_old	Azure	19.71	18.24	92.5 %
EBS volume db-backup-old	AWS	16.00	15.20	95.0 %
Total		**35.71**	**33.44**	**93.6 %**

6.9 Test Results

Table 6.4: Automated test run (pytest)

Suite	Tests	Result
Algorithms (priority, anomaly, forecast, VM verdict, verification)	23	passed
Full cost-to-action loop on a fake Azure cloud	1	passed
Static front end and Azure diagnostics	3	passed
Total	**27**	**27 passed**

The integration test exercises the complete loop including permissions and a regression: after a fix is verified the resource is made to reappear and the test asserts that the finding reopens and the saving is withdrawn.

6.10 Validation on a Real Azure Subscription

The Azure scripts of Section 5.12 are the procedure used to validate the system on a subscription. The expected and observable results are listed in Table 6.5. The figures in Sections 6.2 to 6.8 are from the demonstration workspace; results from a customer's own subscription depend on the resources it contains and on Azure's publication delay of 8 to 24 hours for billing data.

Table 6.5: Validation checklist for a live subscription

Check	Expected observation	When
`python -m app.manage check-azure`	"Connected: <subscription name>"	immediately
Workspace created from the settings	A customer workspace appears next to the demo; the sidebar shows the subscription	at start-up
Planted waste (02-leaks.sh)	Four findings with the signed-in account as creator, no tags	minutes (Activity Log delay)
Cost charts and budget	Daily spend by service for the workload	8 to 24 hours
Anomaly (05-spike.sh up)	A flagged day after about a week of normal days	about 1 week
Fix a finding (delete the leftover disk)	State changes to resolved, then verified after 3 or more billed days	about 4 to 8 days

6.11 Discussion

The results show that the central claim of the project is realised in software: a cost problem is detected, priced, ranked, owned, acted on and checked against the bill, and the numbers on every page agree with each other. Three design choices proved important. First, **fingerprints** keep one finding per problem across syncs, so the history, comments and verification stay attached. Second, **verification against the bill** replaces self-reported savings; in the demonstration it also reveals that estimates are slightly optimistic (94 % accurate), information that a plain report would never produce. Third, **read-only access**

with a managed-identity option makes the tool safe to connect, which is a precondition for any real adoption.

Limitations were also observed. Cost data lags by up to a day, so verification of a fix cannot be instantaneous. Memory pressure on virtual machines is invisible without an agent, so rightsizing advice carries lower confidence. Owners are known only for resources created in the last 89 days unless tags exist. The scheduler and the login rate limiter keep state in one process, so the current version targets a single instance. These points are taken up as future work in the next chapter.

7. CONCLUSION

This project designed, implemented and tested **CloudPulse**, a cloud cost monitoring and optimisation platform that treats a cost problem as a task to be owned and proven fixed, not as a number on a chart. The system connects to an Azure subscription with a read-only service principal, a CLI login or a managed identity, reads the bill, inventory, metrics, recommendations and audit log through a small custom REST client, and converts them into de-duplicated findings. Transparent algorithms, a robust weekday-aware anomaly detector, Holt's smoothing forecast with an 80 % range and a logarithmic priority score, make every figure explainable. Findings are assigned to the creator found in the Activity Log, move through a role-controlled lifecycle, and are counted as savings only after seven days of before-and-after billing confirm the drop. A fixed resource that returns reopens its finding and withdraws the saving.

The objectives of Chapter 2 were met: Azure data collection with least privilege, detection of idle and wasted resources, anomaly and budget forecasting, ownership attribution, lifecycle management with an audit trail, verified savings, secure credential handling and a reproducible Azure test environment. The 27 automated tests passed, and a headless-browser review of all pages found and fixed a presentation defect. In the demonstration workspace the system reported $262.94 a month of open waste and $33.44 a month of verified savings at 94 % estimate accuracy.

7.1 Limitations

- Savings can only be confirmed after Azure publishes the bill, so verification takes about a week.
- Right-sizing uses CPU and network only; memory is not measured.
- Creators older than the 90-day Activity Log retention are known only through tags.
- The scheduler, login throttle and SQLite database make this version a single-instance application.
- AWS and FOCUS support are present but less deep than the Azure integration.

7.2 Future Enhancements

- Sign-in with Microsoft Entra ID (OpenID Connect) so that organisations can use their existing accounts.
- E-mail, Microsoft Teams and Slack notifications in addition to the in-app inbox.

- Google Cloud collector and deeper AWS coverage (idle load balancers, unused NAT gateways, snapshots).
- Reservation and savings-plan coverage analysis and purchase recommendations.
- Tag-compliance policies with owner reminders, and showback or chargeback reports by team.
- Optional, approval-gated remediation, for example deallocating a stopped VM after the owner confirms.
- PostgreSQL as the default store with a shared job queue for multi-instance deployment.
- Memory and disk metrics through the Azure Monitor agent for more accurate right-sizing.

8. REFERENCE

[1] Microsoft, "Azure Cost Management and Billing documentation," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/azure/cost-management-billing/

[2] FinOps Foundation, "What is FinOps?" [Online]. Available: https://www.finops.org/introduction/what-is-finops/

[3] S. Ramírez, "FastAPI framework documentation." [Online]. Available: https://fastapi.tiangolo.com/

[4] SQLAlchemy Project, "SQLAlchemy 2.0 documentation." [Online]. Available: https://docs.sqlalchemy.org/

[5] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," RFC 7914, IETF, Aug. 2016.

[6] Python Cryptographic Authority, "Fernet (symmetric encryption)," cryptography documentation. [Online]. Available: https://cryptography.io/en/latest/fernet/

[7] FinOps Foundation, "FOCUS: FinOps Open Cost and Usage Specification." [Online]. Available: https://focus.finops.org/

[8] B. Iglewicz and D. C. Hoaglin, How to Detect and Handle Outliers, vol. 16 of The ASQC Basic References in Quality Control. Milwaukee, WI, USA: ASQC Quality Press, 1993.

[9] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," Int. J. Forecasting, vol. 20, no. 1, pp. 5-10, 2004 (reprint of the 1957 ONR memorandum).

[10] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[11] D. Hardt, Ed., "The OAuth 2.0 authorization framework," RFC 6749, IETF, Oct. 2012.

[12] Microsoft, "Azure Resource Manager REST API reference," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/rest/api/azure/

[13] Microsoft, "Overview of Azure Resource Graph," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/azure/governance/resource-graph/overview

[14] Microsoft, "Azure Monitor metrics overview," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/azure/azure-monitor/essentials/data-platform-metrics

[15] Microsoft, "Azure activity log," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/azure/azure-monitor/essentials/activity-log

[16] Microsoft, "Azure Advisor documentation," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/azure/advisor/

[17] Microsoft, "Azure Retail Prices REST API," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/rest/api/cost-management/retail-prices/azure-retail-prices

[18] Microsoft, "Azure built-in roles," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/azure/role-based-access-control/built-in-roles

[19] Microsoft, "What are managed identities for Azure resources?" Microsoft Learn. [Online]. Available: https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/overview

[20] J. R. Storment and M. Fuller, Cloud FinOps, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2023.